

DOCUMENTACIÓN TÉCNICA DE LA API: CivilControl BACKEND

Sistema de Gestión Integral para ESEA SA

Información del Proyecto:

- Versión:** 1.0.0
- Última actualización:** 20 de enero de 2026
- Framework:** Spring Boot 3.3.0
- Lenguaje:** Java 17
- Sistemas de Gestión de Base de Datos:** PostgreSQL

1. TABLA DE CONTENIDOS

- Arquitectura General
- Stack Tecnológico
- Estructura del Proyecto
- Módulos de la API
- Seguridad y Autenticación
- Manejo de Excepciones
- Base de Datos
- Endpoints de la API
- Configuración y Despliegue
- Mejores Prácticas Implementadas

2. ARQUITECTURA GENERAL

2.1. Patrón Arquitectónico: Hexagonal (Ports & Adapters)

El sistema se organiza en capas concéntricas para garantizar el desacoplamiento:

- Capa de Presentación (Controllers):** Gestión de APIs REST, validación de entrada y documentación OpenAPI/Swagger.
- Capa de Aplicación (Services - Use Cases):** Implementación de la lógica de negocio, validaciones complejas y coordinación de operaciones.
- Capa de Dominio (Entities & DTOs):** Definición de entidades de negocio, reglas de dominio y mappers (MapStruct).

4. **Capa de Infraestructura (Repositories):** Acceso a datos mediante Spring Data JPA, queries personalizadas y gestión de transacciones.
5. **Capa de Persistencia (Database):** PostgreSQL/MySQL, gestión de migraciones con Flyway y políticas de integridad.

2.2. Características Arquitectónicas

- Separación estricta de responsabilidades por capas.
 - Uso de Interfaces (Ports) y sus respectivas implementaciones (Adapters).
 - Inversión de Dependencias para asegurar que los servicios dependan de abstracciones.
 - Inyección de Dependencias gestionada por Spring Framework.
 - Gestión de Transacciones Declarativas mediante la anotación `@Transactional`.
 - Implementación de Soft Delete (eliminación lógica) a través del flag `deleted`.
-

3. STACK TECNOLÓGICO

3.1. Framework Core

- Spring Boot 3.3.0.
- Spring Data JPA para persistencia y ORM.
- Spring Security para autenticación y autorización.
- Spring Validation para la validación de integridad de datos.

3.2. Seguridad

- JWT (JSON Web Tokens) para autenticación sin estado (stateless).
- BCrypt para la encriptación de credenciales.
- CORS para la configuración de acceso cross-origin.

3.3. Base de Datos y Persistencia

- Hibernate como motor de ORM.
- Flyway para la gestión de migraciones de esquema.
- Soporte para PostgreSQL y MySQL.

3.4. Herramientas de Mapeo y Utilidades

- MapStruct 1.5.5 para el mapeo automático entre Entidades y DTOs.
- Lombok para la reducción de código repetitivo (boilerplate).

3.5. Documentación y Reportes

- SpringDoc OpenAPI 3 y Swagger UI para documentación interactiva.
- Apache POI 5.2.3 para la generación de archivos Excel.

- iText 7.2.5 para la generación de documentos PDF.

3.6. Gestión de Ciclo de Vida

- Maven como gestor de dependencias y herramienta de construcción.
 - Java 17 como entorno de ejecución.
-

4. ESTRUCTURA DEL PROYECTO

El código fuente se organiza bajo la siguiente estructura de directorios:

1. **config/**: Clases de configuración global (CORS, Seguridad, OpenAPI).
 2. **controller/**: Controladores REST que exponen los recursos de la API.
 3. **service/**: Lógica de negocio dividida en interfaces (port) e implementaciones.
 4. **repository/**: Interfaces de Spring Data JPA para el acceso a datos.
 5. **model/**: Definición del modelo de datos, incluyendo:
 - **entity/**: Entidades de persistencia.
 - **dto/**: Objetos de transferencia de datos.
 - **mapper/**: Lógica de transformación entre modelos.
 - **enums/** y **validation/**: Tipos definidos y validaciones personalizadas.
 6. **exception/**: Jerarquía de excepciones personalizadas y manejador global de errores.
 7. **resources/**: Archivos de configuración (`application.properties`) y scripts de migración.
-

5. MÓDULOS DE LA API

5.1. Gestión de Recursos Humanos

- **Employees (Empleados)**: CRUD completo, búsqueda avanzada, validación de DNI/CUIL únicos, soft delete y validación de mayoría de edad.
- **Salary Payments (Pagos de Salario)**: Registro de haberes, cálculo de montos y prevención de pagos duplicados por período.
- **Disciplinary Actions (Acciones Disciplinarias)**: Registro y seguimiento de sanciones (verbales, escritas, suspensiones, despidos).
- **Employee Vacations (Vacaciones)**: Gestión de períodos, validación de disponibilidad y cálculo de días.
- **EPP Deliveries (Entrega de EPP)**: Control de entrega de equipos de protección y registro de stock.

5.2. Gestión de Flota y Vehículos

- **Vehicles (Vehículos):** Gestión de flota, validación de patentes y asignación a proyectos.
- **Fuel Loads (Cargas de Combustible):** Registro de consumos, control de tickets y asociación con estaciones de servicio.
- **Insurance Policies (Pólizas de Seguro):** Seguimiento de vigencia, tipos de cobertura y asociación de vehículos.
- **Repairs y Licence Plate Payments:** Gestión de mantenimiento correctivo/preventivo y pagos de patentes.

5.3. Gestión Financiera y Proveedores

- **Suppliers (Proveedores):** Registro de proveedores, validación de CUIT, gestión de saldos y métodos de pago.
- **Transactional Documents:** Administración de facturas, notas de crédito y débito con numeración automática.
- **Payments:** Registro de pagos mediante efectivo, transferencia o cheque.

5.4. Gestión de Inventario, Proyectos y Seguridad

- **Inventory:** Control de stock por edificio (almacenes/oficinas) con alertas de existencias mínimas.
- **Project Areas:** Organización de recursos (empleados y vehículos) asignados a áreas específicas de trabajo.
- **Security:** Administración de usuarios, roles y permisos granulares mediante encriptación BCrypt y tokens JWT.

6. SEGURIDAD Y AUTENTICACIÓN

6.1. Modelo JWT

El proceso de autenticación sigue el flujo estándar:

1. Autenticación del usuario mediante credenciales.
2. Generación y entrega de un token JWT firmado.
3. Inclusión del token en el encabezado **Authorization: Bearer <token>** para solicitudes protegidas.
4. Validación de firma y expiración en cada solicitud mediante el filtro de seguridad.

6.2. Estructura de Permisos

Los permisos se definen de forma granular siguiendo el esquema: **MODULO_ACCION** (Ejemplo: **EMPLOYEE_CREATE**, **VEHICLE_READ**).

7. MANEJO DE EXCEPCIONES

El sistema implementa una estrategia de gestión de errores en cuatro niveles:

1. **Validación Preventiva:** Verificaciones en la capa de servicio antes del procesamiento.
 2. **Captura en Persistencia:** Gestión de errores a nivel de base de datos (Ej. `DataIntegrityViolationException`).
 3. **Análisis de Errores:** Identificación inteligente del campo en conflicto para proporcionar feedback preciso.
 4. **Respuesta HTTP:** Mapeo de excepciones a códigos de estado estándar (200, 201, 400, 401, 403, 404, 409, 500).
-

8. BASE DE DATOS

8.1. Diseño de Datos

- **Auditoría:** Todas las entidades incluyen marcas temporales de creación y actualización.
- **Integridad:** Constraints aplicadas tanto a nivel de aplicación (Java Bean Validation) como en el motor de base de datos.
- **Optimización:** Uso de índices en columnas de búsqueda recurrente y claves únicas.

8.2. Migraciones

Se utiliza **Flyway** para el control de versiones del esquema, asegurando que todos los entornos (desarrollo, testing, producción) mantengan la misma estructura de datos.

9. CONFIGURACIÓN Y DESPLIEGUE

9.1. Perfiles de Ejecución

- **Development (**dev**):** Configuración para bases de datos locales y niveles de log detallados (DEBUG).
- **Production (**prod**):** Configuración optimizada, logs restringidos (WARN) y uso de variables de entorno para datos sensibles.

9.2. Procedimiento de Construcción

1. Compilación: `./mvnw clean package -DskipTests`
2. Ejecución: `java -jar -Dspring.profiles.active=prod app.jar`

10. MEJORES PRÁCTICAS IMPLEMENTADAS

1. **Código Limpio:** Aplicación de principios SOLID y Single Responsibility.
 2. **Seguridad:** Sanitización de entradas, encriptación de datos sensibles y protocolos de autenticación seguros.
 3. **Rendimiento:** Implementación de Lazy Loading en relaciones JPA y paginación obligatoria en todos los endpoints de consulta de listas.
 4. **Mantenibilidad:** Desacoplamiento total mediante el uso de DTOs y arquitectura en capas.
-

11. IMPLEMENTACIÓN DE INTERNACIONALIZACIÓN (i18n)

11.1. Arquitectura y Componentes Core

- **InternationalizationConfig:** Clase de configuración ubicada en `config/` que define los beans `MessageSource` y `LocaleResolver`. Establece `es_AR` como locale por defecto y optimiza el rendimiento con un caché de 3600 segundos .
- **MessageSourceHelper:** Utilidad centralizada en `service/util/` para el acceso a mensajes traducidos. Implementa un modo dual de uso: inyección de dependencias para servicios y métodos estáticos para la jerarquía de excepciones .
- **ApplicationContextProvider:** Provee acceso estático al contexto de Spring, permitiendo que las excepciones personalizadas resuelvan mensajes sin estar gestionadas directamente por el contenedor de beans .
- **messages_es.properties:** Repositorio central de recursos en `src/main/resources/` con más de 180 mensajes parametrizados para todas las entidades del sistema .

11.2. Patrones de Integración Técnica

- **Validación en DTOs:** Uso de anotaciones de Bean Validation (`@NotBlank`, `@Size`, etc.) vinculadas directamente a llaves de internacionalización para una respuesta de error estandarizada .
- **Servicios y Excepciones:** Migración del 100% de las validaciones hardcoded a un modelo basado en llaves (`entity.field.validation`), permitiendo mensajes dinámicos mediante el uso de placeholders `{0}`.
- **Política de Codificación:** Implementación estricta de UTF-8 con la convención técnica de omitir caracteres acentuados en los archivos de propiedades para garantizar la compatibilidad total entre entornos de despliegue.

11.3. Métricas de Cobertura (Finalizado Enero 2026)

- **Módulos Migrados:** 23 de 23 servicios operativos (100%), incluyendo Seguridad, Flota, Inventario y Finanzas.
 - **Excepciones Personalizadas:** 12 de 12 clases de excepción (100%) adaptadas al sistema de mensajería dinámica.
 - **Estado del Arte:** Eliminación total de strings hardcoded en la lógica de negocio, asegurando un sistema escalable y preparado para la incorporación de nuevos idiomas (ej. `messages_en.properties`).
-

Desarrollado y mantenido por: Andriola Maximo

Fecha de actualización: 20 de enero de 2026